마이크로프로세서

Chapter 3. Hardware Abstraction Layer

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY

Hardware Abstraction Layer

>> What is HAL

- Hardware Abstraction Layer(HAL)는 하드웨어 추상화 계층으로, 하드웨어의 세부 사항을 숨기고 프로그램이 하드웨어와 독립적으로 동작하도록 도와줌
- 다양한 하드웨어에서 일관된 인터페이스를 제공하는 목적
- STM32 마이크로프로세서의 HAL은 다양한 주변장치(Peripheral)에 대한 추상화를 통해 사용자가 직접적으로 하드웨어 레지스터를 제어할 필요성을 낮춤
- HAL은 STMF0부터 H7까지 다양한 프로세서에 대해 넓은 호환성을 제공하여, 서로 다른 프로세서 사이의 쉬운 코드 이식을 도와줌
- CubeMX를 통한 연동을 제공하여 손쉽게 설정을 적용하고 코드 생성이 가능함

Hardware Abstraction Layer

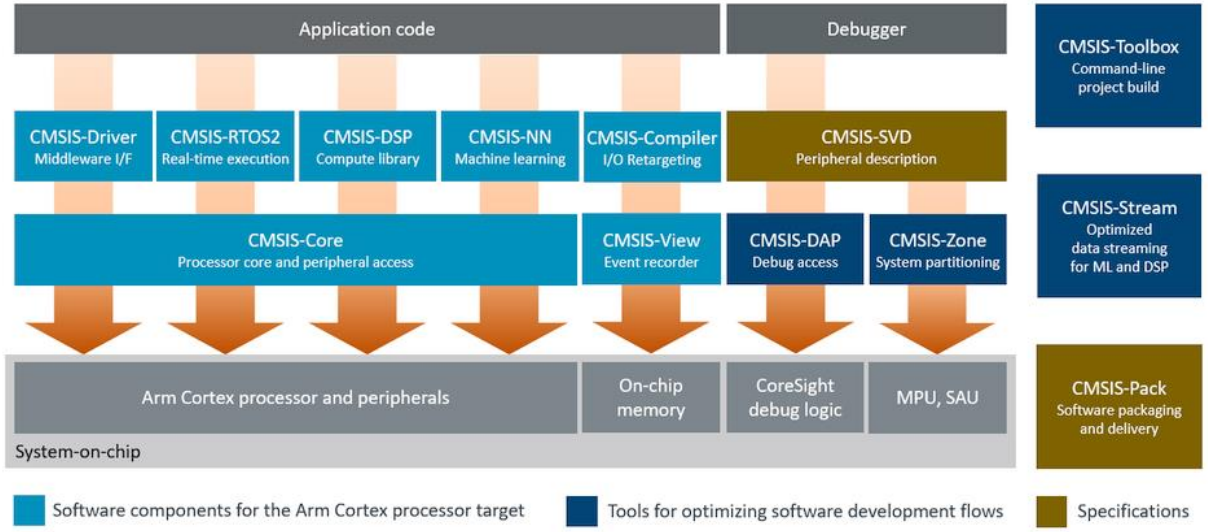
>> HAL & CMSIS

- Cortex Microcontroller Software Interface Standard(CMSIS)
- CMSIS는 ARM Cortex-M 프로세서 기반의 마이크로컨트롤러를 위한 소프트웨어 인터페이스 표준
- CMSIS 기반으로 HAL을 구축하여 높은 개발 효율성을 추구함
- HAL은 CMSIS를 통해 Cortex-M 프로세서의 주요 기능 관리와 더불어 주변장치(Peripheral) 제어, 실시간 운영체제(RTOS) 통합을 지원함

Hardware Abstraction Layer

>> Structure of CMSIS

- CMSIS-Core: Cortex-M 프로세서 코어와 주변장치에 대한 API. 시스템 초기화, 인터럽트 관리, 프로세서 레지스터, 클럭 설정 등을 포함
- CMSIS-Driver: 주변장치와 외부 장치에 대한 표준 인터페이스 제공
- CMSIS-DSP: Cortex-M 프로세서에 최적화된 신호 처리 라이브러리 제공. FFT, 행렬 연산, 고정 및 부동 소수점 연산 라이브러리 제공
- CMSIS-RTOS: Real-Time Operation System을 위한 공통 API



CMSIS COMPLIANT
ARM® Cortex™ Microcontroller Software Interface Standard

KYUNG HEE UNIVERSITY

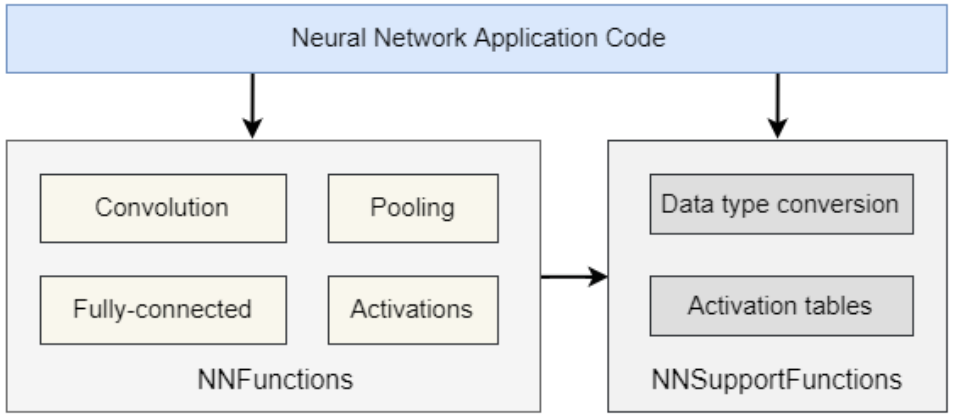
Hardware Abstraction Layer

>> Structure of CMSIS

- CMSIS-NN: 임베디드 시스템에서 신경망 구현을 위한 최적화 신경망 커널 모음 제공

- CMSIS-DAP: Debug Access Port. 디버그 유닛에 대한 표준 펌웨어

- CMSIS-SVD: System View Description. 시스템을 표준화하여 XML형식으로 기술. 디버깅과정에서 하드웨어를 효율적으로 인식도록 도와줌

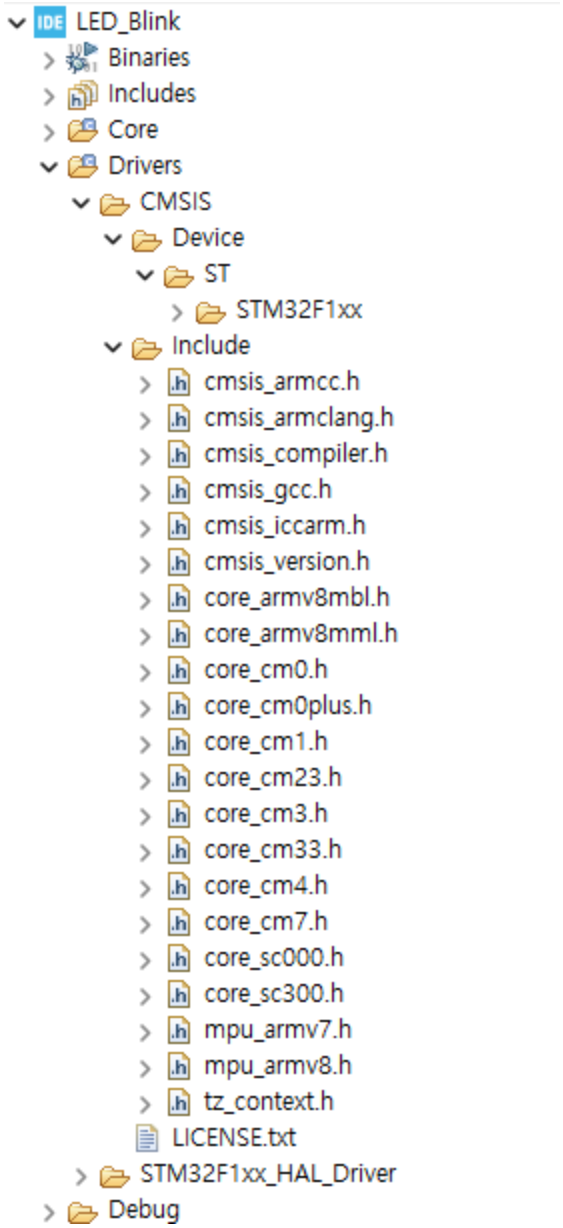


CMSIS-NN Block Diagram

```
PS C:\ST\STM32CubeCLT_1.16.0\STMicroelectronics_CMSIS_SVD> ls


    디렉터리: C:\ST\STM32CubeCLT_1.16.0\STMicroelectronics_CMSIS_SVD


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----       2024-09-04   오후 3:23               Core
-a----       2024-06-29   오전 2:43           6053 about.html
-a----       2024-06-29   오전 2:41        2308462 STM32C011.svd
-a----       2024-06-29   오전 2:41        2323563 STM32C031.svd
-a----       2024-06-29   오전 2:41        3059976 STM32C071.svd
-a----       2024-06-29   오전 2:41         564272 STM32F0x0.svd
-a----       2024-06-29   오전 2:41        1099190 STM32F0x1.svd
-a----       2024-06-29   오전 2:41        1098557 STM32F0x2.svd
-a----       2024-06-29   오전 2:41        1101474 STM32F0x8.svd
-a----       2024-06-29   오전 2:41         487395 STM32F100.svd
-a----       2024-06-29   오전 2:41        1259067 STM32F101.svd
-a----       2024-06-29   오전 2:41         999511 STM32F102.svd
-a----       2024-06-29   오전 2:41        1060322 STM32F103.svd
-a----       2024-06-29   오전 2:41        1382987 STM32F107.svd
```



Hardware Abstraction Layer

>> Structure of HAL

- **주변 장치 핸들 구조체**
  주변 장치의 기본적인 설정, 레지스터 등을 관리

- **초기화 및 동작 조건 설정 구조체**
  주변 장치의 초기 설정 및 동작 조건 정의

- **작업 수행 구조체**
  특정 작업 수행을 위해 함수 내에서 활용

- **주변 장치 인터럽트 핸들러**
  주변 장치에서 발생하는 인터럽트 처리

- **사용자 콜백 함수**
  인터럽트 발생시 호출되는 사용자 정의 함수

```
> (anonymous)
  TIM_HandleTypeDef : struct

> (anonymous)
  RCC_PLLInitTypeDef : struct
> (anonymous)
  RCC_ClkInitTypeDef : struct
```

```c
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  /* EXTI line interrupt detected */
  if (__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00u)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);
    HAL_GPIO_EXTI_Callback(GPIO_Pin);
  }
}
```

```c
__weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
  /* Prevent unused argument(s) compilation warning */
  UNUSED(GPIO_Pin);
  /* NOTE: This function Should not be modified, when the callback is needed,
           the HAL_GPIO_EXTI_Callback could be implemented in the user file
   */
}
```

Hardware Abstraction Layer

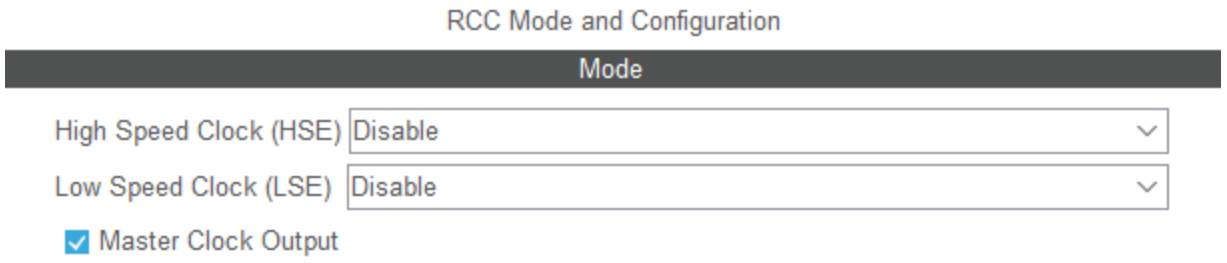
>> Type of HAL Drivers

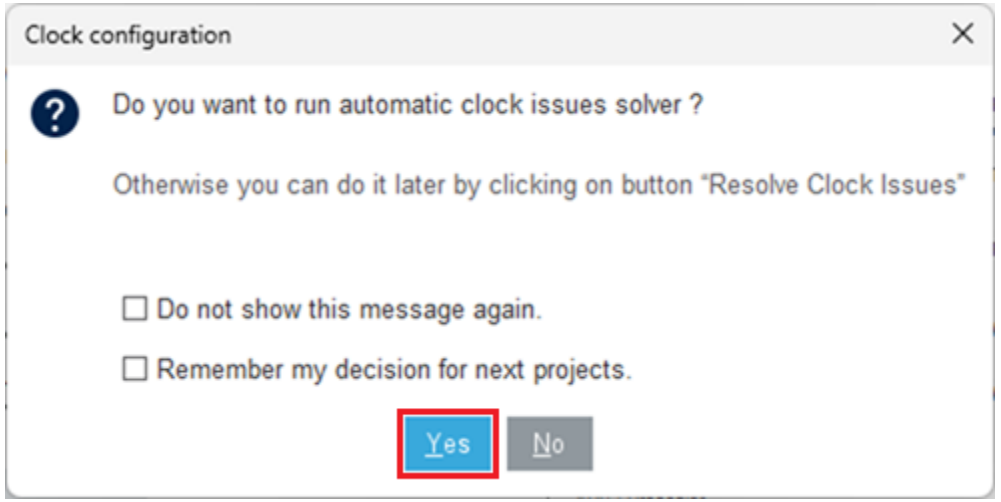
- HAL Cortex Drivers
- HAL GPIO Drivers
- HAL UART Drivers
- HAL I2C Drivers
- HAL SPI Drivers
- HAL CAN Drivers
- HAL TIM Drivers
- HAL ADC/DAC Drivers
- HAL IWDG/WWDG Drivers
- HAL RCC Drivers
- HAL RTC Drivers
- HAL DMA Drivers

Reset Clock Controller

>> RCC, System Clock

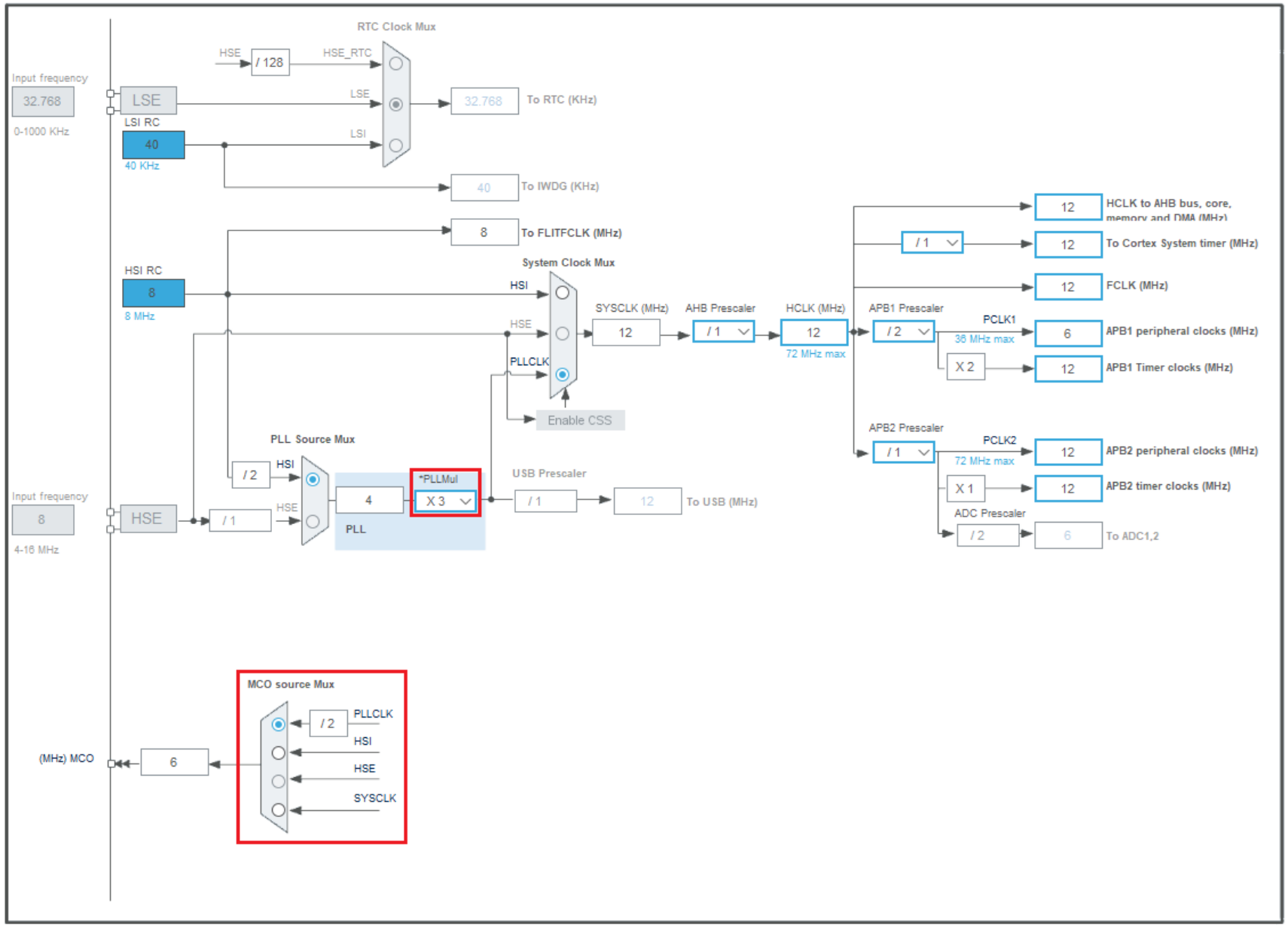
- 새 프로젝트 생성 및 RCC 설정
- Master Clock Output(MCO) 활성화
- MCO 핀 출력 제한으로 인한 클럭 설정 경고
- PA8 핀을 RCC_MCO로 자동 설정





Reset Clock Controller

>> RCC, System Clock



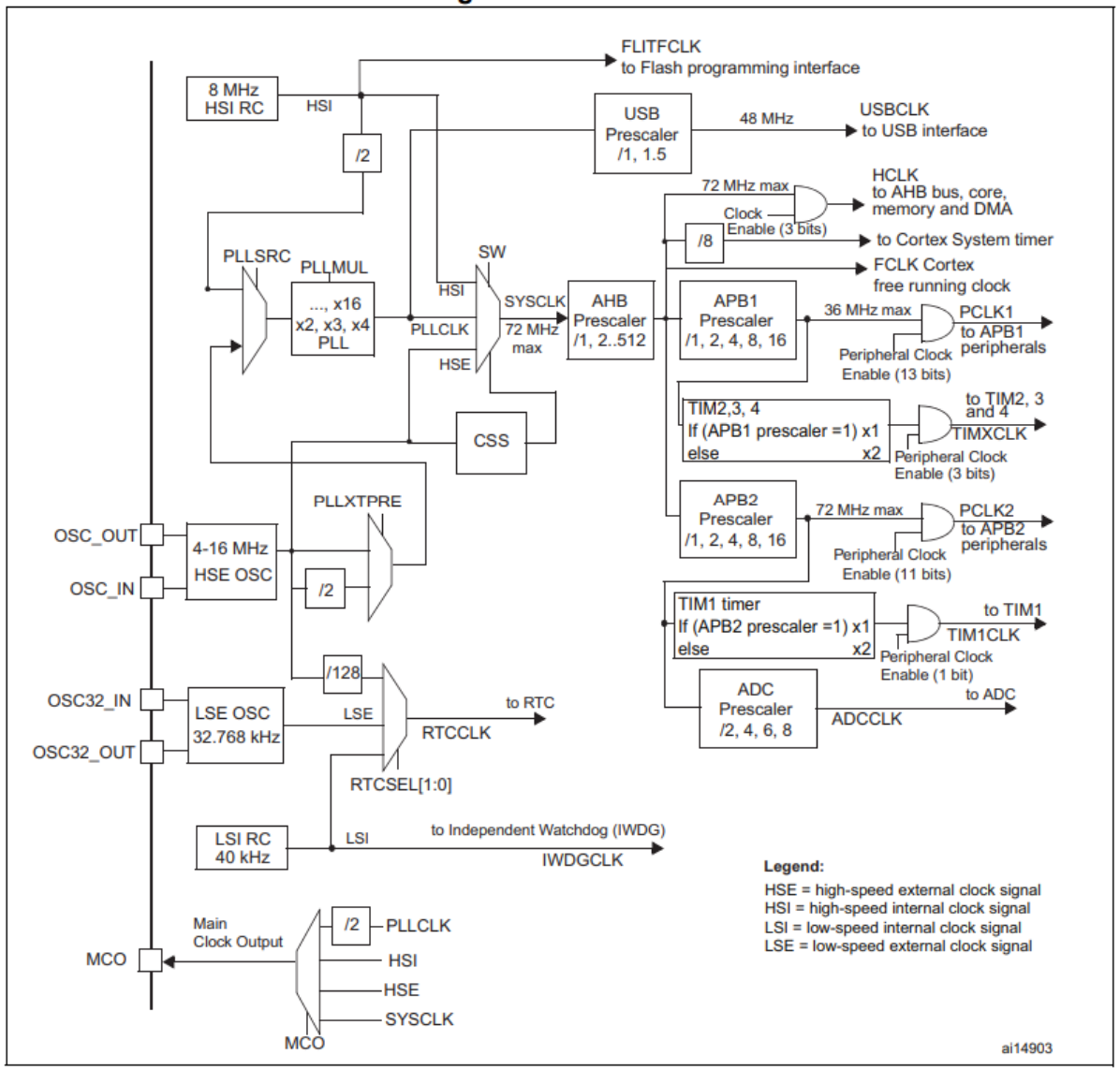
KYUNG HEE UNIVERSITY

Reset Clock Controller

>> RCC Clock Tree

- High Speed Internal(HSI) 8MHz
- Phase-Locked Loop, 입력 클럭 신호 변경, 주파수 합성 회로
- HSE 혹은 HSI/2가 PLL Source(PLLSRC)로 선택
- PLL MUX에서 2~16배수로 클럭을 변경(PLLCLK)
- System Clock Mux에서 HSI, PLLCLK, HSE에서 주 시스템 클럭(SYSCLK)를 선택
- AHB Prescaler를 통해 시스템 클럭을 조정하여 주변 장치에서 활용(저전력, 속도 최적화, 통신 속도 조절...)

Figure 2. Clock tree



Reset Clock Controller

>> RCC Register

- RCC 레지스터 설정을 통해 프로세서의 클럭 설정
- Reference Manual을 통해 해당 레지스터 확인

7.3.1 Clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x0000 XX83 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved						PLL RDY	PLLON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
						r	rw					rw	rw	r	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

7.3.2 Clock configuration register (RCC_CFGR)

Address offset: 0x04

Reset value: 0x0000 0000

Access: 0 ≤ wait state ≤ 2, word, half-word and byte access

1 or 2 wait states inserted only if the access occurs during clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					MCO[2:0]			Res.	USB PRE	PLLMUL[3:0]				PLL XTPRE	PLL SRC
					rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADCPRE[1:0]		PPRE2[2:0]			PPRE1[2:0]			HPRE[3:0]				SWS[1:0]		SW[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r	rw	rw

0x4002 1000 - 0x4002 13FF	Reset and clock control RCC

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	RCC_CR	Reserved						PLL RDY	PLL ON	Reserved				CSSON	HSEBYP	HSERD	HSEON	HSICAL[7:0]								HSITRIM[4:0]					Reserved	HSIRDY	HSION
	Reset value							0	0					0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0		1	1
0x04	RCC_CFGR	Reserved					MCO [2:0]			Reserved	USBPRE	PLLMUL [3:0]				PLLXTPRE	PLLSRC	ADC PRE [1:0]		PPRE2 [2:0]			PPRE1 [2:0]			HPRE[3:0]				SWS [1:0]		SW [1:0]	
	Reset value						0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	RCC_CIR	Reserved								CSSC	Reserved	PLLRDYC	HSERDYC	HSIRDYC	LSERDYC	LSIRDYC	Reserved				PLLRDYIE	HSERDYIE	HSIRDYIE	LSERDYIE	LSIRDYIE	CSSF	Reserved			PLLRDYF	HSERDYF	HSIRDYF	LSERDYF	LSIRDYF
	Reset value									0		0	0	0	0	0					0	0	0	0	0	0				0	0	0	0	0
0x0C	RCC_APB2RSTR	Reserved										TIM11RST	TIM10RST	TIM9RST	Reserved			ADC3RST	USART1RST	TIM8RST	SPI1RST	TIM1RST	ADC2RST	ADC1RST	IOPGRST	IOPFRST	IOPERST	IOPDRST	IOPCRST	IOPBRST	IOPARST	Reserved	AFIORST	
	Reset value											0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	
0x010	RCC_APB1RSTR	Reserved			DACRST	PWRRST	BKPRST	Reserved	CANRST	Reserved	USBRST	I2C2RST	I2C1RST	UART5RST	UART4RST	USART3RST	USART2RST	Reserved	SPI3RST	SPI2RST	Reserved		WWDGRST	Reserved		TIM14RST	TIM13RST	TIM12RST	TIM7RST	TIM6RST	TIM5RST	TIM4RST	TIM3RST	TIM2RST
	Reset value				0	0	0		0		0	0	0	0	0	0	0		0	0			0			0	0	0	0	0	0	0	0	0
0x14	RCC_AHBENR	Reserved																						SDIOEN	Reserved	FSMCEN	Reserved	CRCEN	Reserved	FLITFEN	Reserved	SRAMEN	DMA2EN	DMA1EN
	Reset value																							0		0		0		1		1	0	0
0x18	RCC_APB2ENR	Reserved										TIM11 EN	TIM10 EN	TIM9 EN	Reserve d			ADC3EN	USART1EN	TIM8EN	SPI1EN	TIM1EN	ADC2EN	ADC1EN	IOPGEN	IOPFEN	IOPEEN	IOPDEN	IOPCEN	IOPBEN	IOPAEN	Reserved	AFIOEN	
	Reset value											0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	
0x1C	RCC_APB1ENR	Reserved			DACEN	PWREN	BKPEN	Reserved	CANEN	Reserved	USBEN	I2C2EN	I2C1EN	UART5EN	UART4EN	USART3EN	USART2EN	Reserved	SPI3EN	SPI2EN	Reserved		WWDGEN	Reserved		TIM14EN	TIM13EN	TIM12EN	TIM7EN	TIM6EN	TIM5EN	TIM4EN	TIM3EN	TIM2EN
	Reset value				0	0	0		0		0	0	0	0	0	0	0		0	0			0			0	0	0	0	0	0	0	0	0
0x20	RCC_BDCR	Reserved																BDRST	RTCEN	Reserved				RTC SEL [1:0]		Reserved						LSEBYP	LSERDY	LSEON
	Reset value																	0	0					0	0							0	0	0
0x24	RCC_CSR	LPWRSTF	WWDGRSTF	IWDGRSTF	SFTRSTF	PORRSTF	PINRSTF	Reserved	RMVF	Reserved																						LSIRDY	LSION	
	Reset value	0	0	0	0	1	1		0																							0	0	

Reset Clock Controller

>> RCC Register

- Clock control Register
- 25: PLL 클럭 상태 레지스터
- 26: PLL Enable 레지스터, PLL 사용 여부 설정
- [19:3]: HSE 설정 관련 레지스터, STM32F103RB 에는 HSE가 장착되지 않아 사용하지 않음
- 1: HSI 상태 레지스터
- 0: HSI Enable 레지스터

7.3.1 Clock control register (RCC_CR)

Address offset: 0x00

Reset value: 0x0000 XX83 where X is undefined.

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved						PLL RDY	PLLON	Reserved				CSS ON	HSE BYP	HSE RDY	HSE ON
						r	rw					rw	rw	r	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HSICAL[7:0]								HSITRIM[4:0]					Res.	HSI RDY	HSION
r	r	r	r	r	r	r	r	rw	rw	rw	rw	rw		r	rw

Bit 25 **PLLRDY:** PLL clock ready flag
Set by hardware to indicate that the PLL is locked.
0: PLL unlocked
1: PLL locked

Bit 24 **PLLON:** PLL enable
Set and cleared by software to enable PLL.
Cleared by hardware when entering Stop or Standby mode. This bit can not be reset if the PLL clock is used as system clock or is selected to become the system clock.
0: PLL OFF
1: PLL ON

Bit 1 **HSIRDY:** Internal high-speed clock ready flag
Set by hardware to indicate that internal 8 MHz RC oscillator is stable. After the HSION bit is cleared, HSIRDY goes low after 6 internal 8 MHz RC oscillator clock cycles.
0: internal 8 MHz RC oscillator not ready
1: internal 8 MHz RC oscillator ready

Bit 0 **HSION:** Internal high-speed clock enable
Set and cleared by software.
Set by hardware to force the internal 8 MHz RC oscillator ON when leaving Stop or Standby mode or in case of failure of the external 4-16 MHz oscillator used directly or indirectly as system clock. This bit cannot be reset if the internal 8 MHz RC is used directly or indirectly as system clock or is selected to become the system clock.
0: internal 8 MHz RC oscillator OFF
1: internal 8 MHz RC oscillator ON

Reset Clock Controller

>> RCC Register

- Clock configuration register
- [26:24]: Microcontroller Clock Output, 지정한 클럭을 MCO 핀으로 출력. SYSCLK, HSI, HSE, PLL/2 선택
- [21:18]: PLL 배수 설정 레지스터, PLL을 통해 변경할 클럭 배수를 설정, x2~x16
- 16: PLL 입력 클럭 설정 레지스터, HSI/2와 HSE 선택
- [1:0]: SYSCLK 설정 레지스터, HSI, HSE, PLL 선택

Bits 1:0 **SW:** System clock switch

Set and cleared by software to select SYSCLK source.
Set by hardware to force HSI selection when leaving Stop and Standby mode or in case of failure of the HSE oscillator used directly or indirectly as system clock (if the Clock Security System is enabled).
00: HSI selected as system clock
01: HSE selected as system clock
10: PLL selected as system clock
11: not allowed

7.3.2 Clock configuration register (RCC_CFGR)

Address offset: 0x04

Reset value: 0x0000 0000

Access: 0 ≤ wait state ≤ 2, word, half-word and byte access

1 or 2 wait states inserted only if the access occurs during clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved					MCO[2:0]			Res.	USB PRE	PLLMUL[3:0]				PLL XTPRE	PLL SRC
					rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADCPRE[1:0]		PPRE2[2:0]			PPRE1[2:0]			HPRE[3:0]				SWS[1:0]		SW[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	r	r	rw	rw

Bits 26:24 **MCO:** Microcontrollerclock output

Set and cleared by software.
0xx: No clock
100: System clock (SYSCLK) selected
101: HSI clock selected
110: HSE clock selected
111: PLL clock divided by 2 selected

Note: This clock output may have some truncated cycles at startup or during MCO clock source switching.

When the System Clock is selected to output to the MCO pin, make sure that this clock does not exceed 50 MHz (the maximum IO speed).

Bits 21:18 **PLLMUL:** PLL multiplication factor

These bits are written by software to define the PLL multiplication factor. These bits can be written only when PLL is disabled.
Caution: The PLL output frequency must not exceed 72 MHz.
0000: PLL input clock x 2
0001: PLL input clock x 3
0010: PLL input clock x 4
0011: PLL input clock x 5
0100: PLL input clock x 6
0101: PLL input clock x 7
0110: PLL input clock x 8
0111: PLL input clock x 9
1000: PLL input clock x 10
1001: PLL input clock x 11
1010: PLL input clock x 12
1011: PLL input clock x 13
1100: PLL input clock x 14
1101: PLL input clock x 15
1110: PLL input clock x 16
1111: PLL input clock x 16

Reset Clock Controller

>> RCC Register

- RCC 레지스터 설정 코드 예시

```c
#ifndef _RCC_H
#define _RCC_H

#define RCC_ADDRESS 0x40021000

#define RCC_CR          *(unsigned long*)   (RCC_ADDRESS+0x00)
#define RCC_CFGR        *(unsigned long*)   (RCC_ADDRESS+0x04)
#define RCC_CIR         *(unsigned long*)   (RCC_ADDRESS+0x08)
#define RCC_APB2RSTR    *(unsigned long*)   (RCC_ADDRESS+0x0C)
#define RCC_APB1RSTR    *(unsigned long*)   (RCC_ADDRESS+0x01)
#define RCC_AHBENR      *(unsigned long*)   (RCC_ADDRESS+0x14)
#define RCC_APB2ENR     *(unsigned long*)   (RCC_ADDRESS+0x18)
#define RCC_APB1ENR     *(unsigned long*)   (RCC_ADDRESS+0x1C)
#define RCC_BDCR        *(unsigned long*)   (RCC_ADDRESS+0x20)
#define RCC_CSR         *(unsigned long*)   (RCC_ADDRESS+0x24)
#define RCC_AHBRSTR     *(unsigned long*)   (RCC_ADDRESS+0x28)
#define RCC_CFGR2       *(unsigned long*)   (RCC_ADDRESS+0x2C)
#define RCC_CFGR2       *(unsigned long*)   (RCC_ADDRESS+0x30)

#define RCC_CR_PLLRDY   (1 << 25)
#define RCC_CR_PLLON    (1 << 24)
#define RCC_CR_HSERDY   (1 << 17)
#define RCC_CR_HSEON    (1 << 16)
#define RCC_CR_HSIRDY   (1 << 1)
#define RCC_CR_HSION    (1 << 0)

#define PRESCALERDIV_NONE   0   // 000
#define PRESCALERDIV_2      4   // 100
#define PRESCALERDIV_4      5   // 101

#define PLLMASK 0xF73D8003 // 1111 0111 0011 1101 1000 0000 0000 0011
#define PLLNODIV    1 // 1       [31] No PLL divide before MCO
#define MCOPRE      0 // 000     [30:28] MCO divide by 1
#define MCO         4 // 100     [26:24] SYSCLK
#define PLLMUL      2 // 0010    [21:18] x4
#define PLLSRC      0 // 00      [16:15] HSI/2
#define SW          2 // 10      [1:0] PLL as system clk

void set_clock();

#endif
```

```c
#include <rcc.h>
#include <stdint.h>
#define DMB() asm volatile ("dmb")

void set_clock() {
    RCC_CR |= RCC_CR_HSION; // HSI Activate 8MHz
    DMB();
    while (!(RCC_CR & RCC_CR_HSIRDY)); // HSI Ready

    uint32_t reg32;

    reg32 = RCC_CFGR;
    reg32 &= ~0xF0; // RCC_CFGR[7:4] HPRE
    RCC_CFGR = (reg32 | (PRESCALERDIV_NONE << 4)); // HPRE set DIV_NONE
    DMB();
    reg32 = RCC_CFGR;
    reg32 &= ~0x3800; // RCC_CFGR[13:11] PPRE2
    RCC_CFGR = (reg32 | (PRESCALERDIV_2 << 11)); // PPRE2 set DIV_NONE
    DMB();
    reg32 = RCC_CFGR;
    reg32 &= ~0x700; // RCC_CFGR[10:8] PPRE1
    RCC_CFGR = (reg32 | (PRESCALERDIV_NONE << 8)); // PPRE1 set DIV_NONE
    DMB();
    reg32 = RCC_CFGR;
    reg32 &= PLLMASK; // PLL Setting Mask
    RCC_CFGR = (reg32 | (PLLNODIV << 31) | (MCOPRE << 28) | (MCO << 24) | (PLLMUL << 18) | (PLLSRC << 15) | (SW << 0)); // PLL Setting
    DMB();

    RCC_CR |= RCC_CR_PLLON;
    DMB();
    while (!(RCC_CR & RCC_CR_PLLRDY));
}
```

Reset Clock Controller

>> RCC HAL Driver

- main.c 시작 부분 SystemClock_Config를 통해 클럭 설정 적용
- CubeMX 상에서 설정한 값을 기반으로 코드 생성
- PLLMUX: x3

main.c

```
RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL3;
```

- MCO Source MUX: PLL/2

main.c

```
HAL_RCC_MCOConfig(RCC_MCO, RCC_MCO1SOURCE_PLLCLK, RCC_MCODIV_1);
```

stm32f1xx_hal_rcc_ex.h

```
#define RCC_MCO1SOURCE_PLLCLK ((uint32_t)RCC_CFGR_MCO_PLLCLK_DIV2)
```

stm32f103xb.h

```
#define RCC_CFGR_MCO_PLLCLK_DIV2 0x07000000U
```

```
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI_DIV2;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL3;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
  HAL_RCC_MCOConfig(RCC_MCO, RCC_MCO1SOURCE_PLLCLK, RCC_MCODIV_1);
}
```

Reset Clock Controller

>> RCC setting: Register vs HAL Driver

- 레지스터를 통해 직접 설정할 경우 보다 세밀하게 설정과 최적화가 가능
- 설정 과정이 복잡하고, 하드웨어에 대한 깊은 이해가 필요함
- 반면 HAL Driver를 이용할 경우 보다 직관적이고 쉽게 설정이 가능함
- 개발 효율성이 높고, 유지보수가 쉬우며 다른 프로세서로의 코드 이식성이 높음

Reset Clock Controller

>> RCC MCO

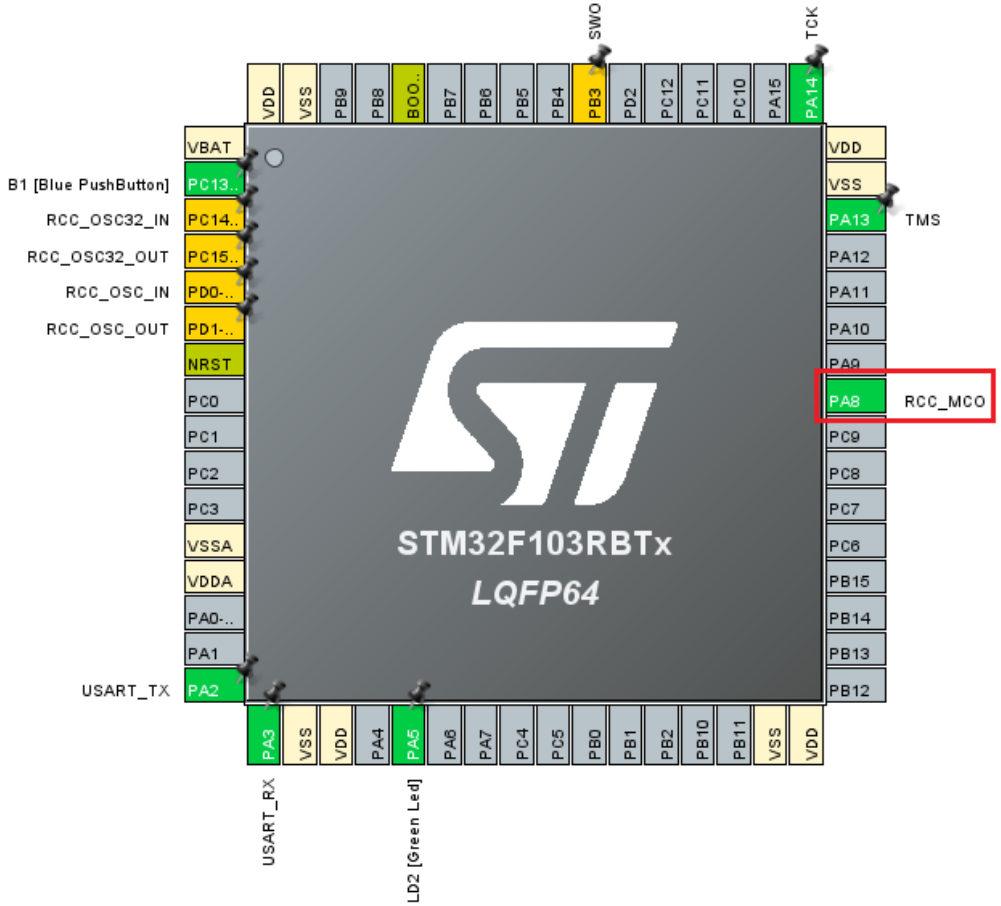
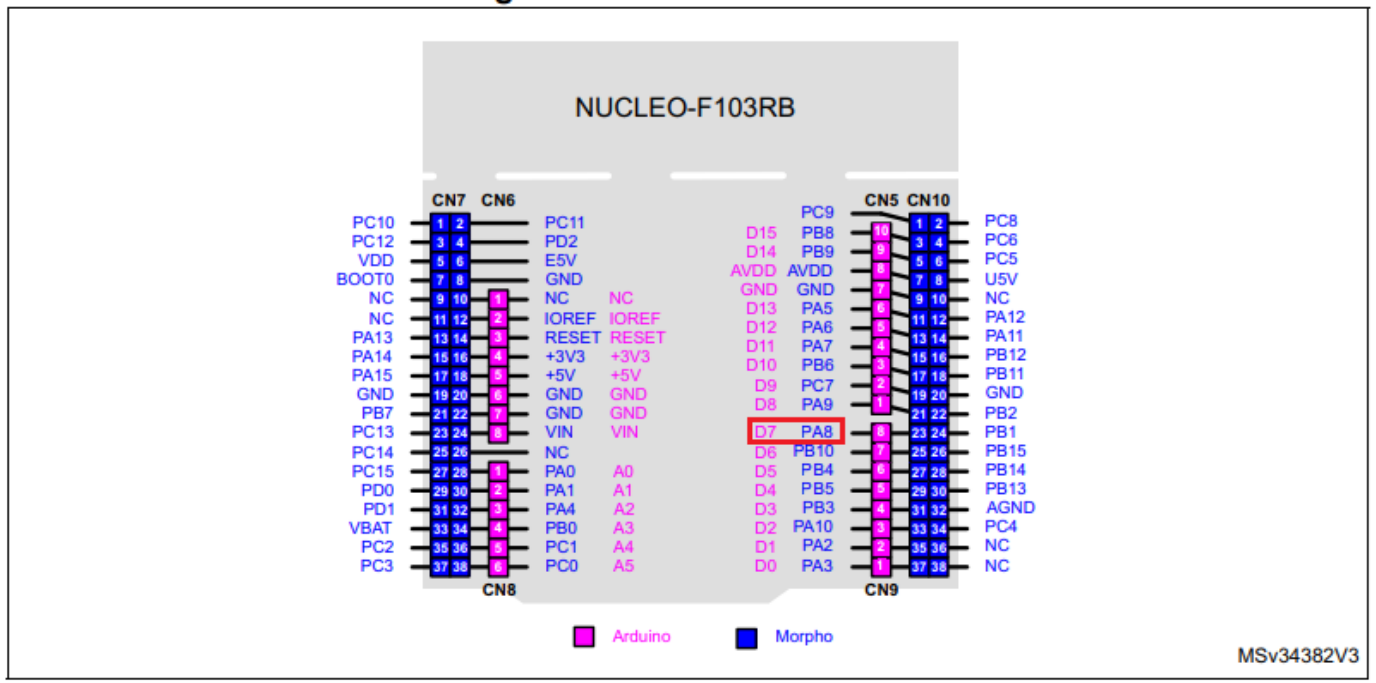


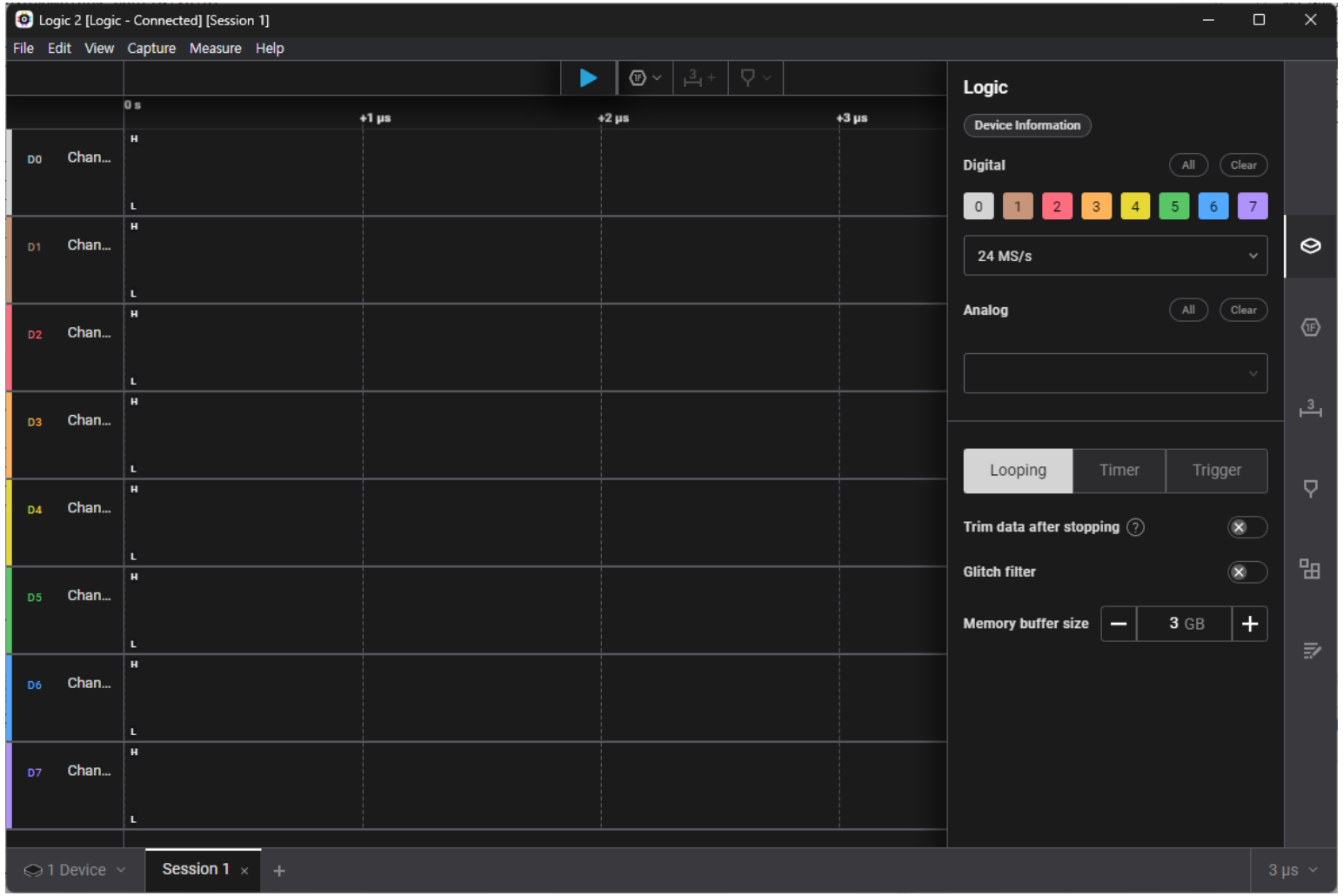
Figure 14. NUCLEO-F103RB



Reset Clock Controller

>> RCC MCO – Logic Analyzer

- 분석기 CH1 – 보드 PA8 연결
- 분석기 GND – 보드 GND 연결
- 로직 분석기 창에서 Start 누를시 기록 시작
- 실습에 사용하는 분석기는 최대 24MS/s 샘플링 가능(초당 2400만 샘플 기록)
- 최대 12MHz까지 측정 가능 (Nyquist)

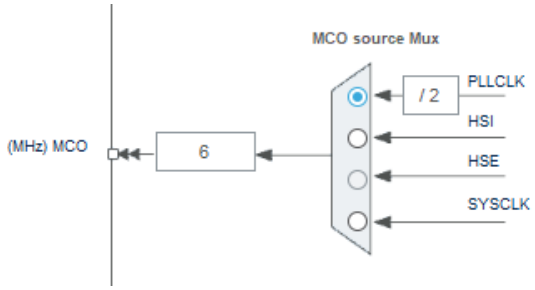


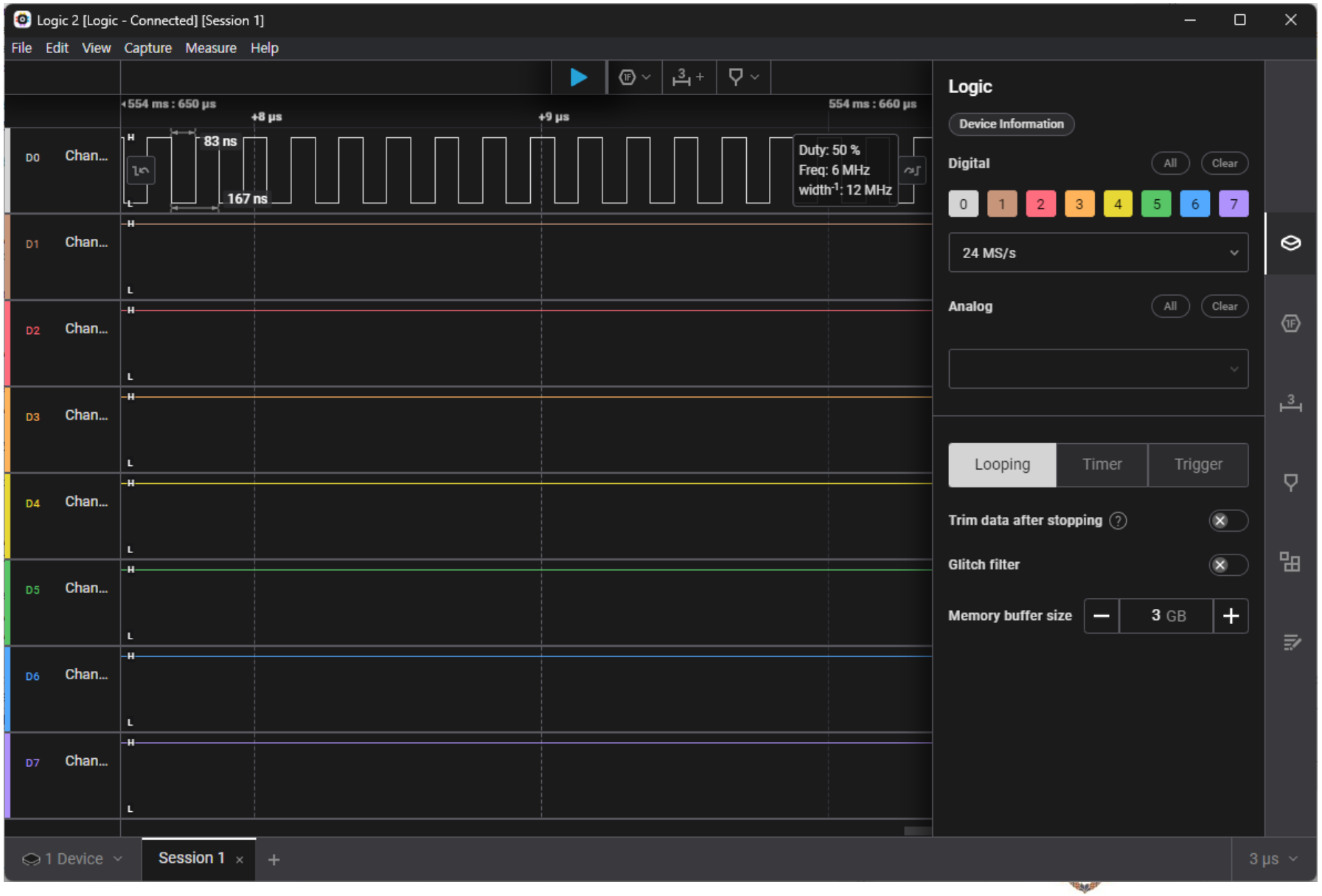
KYUNG HEE UNIVERSITY

Reset Clock Controller

>> RCC MCO – Logic Analyzer

- Ctrl + 드래그로 줌 영역 지정 가능 혹은 마우스 휠로 줌/줌아웃
- MCO 핀으로 출력으로 설정 하였던 6MHz가 출력되는지 확인





Logic Analyzer

>> Logic Analyzer

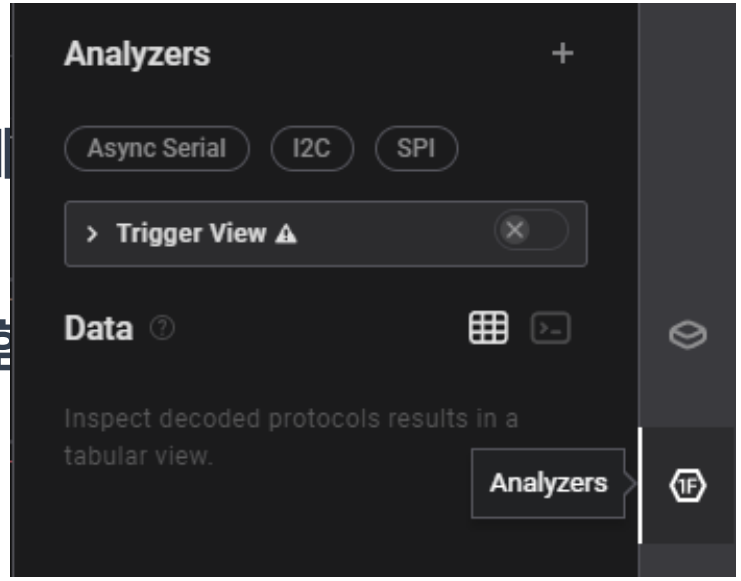
- 로직 분석기는 오실로스코프와 비슷하지만 디지털 신호 측정 및 분석에

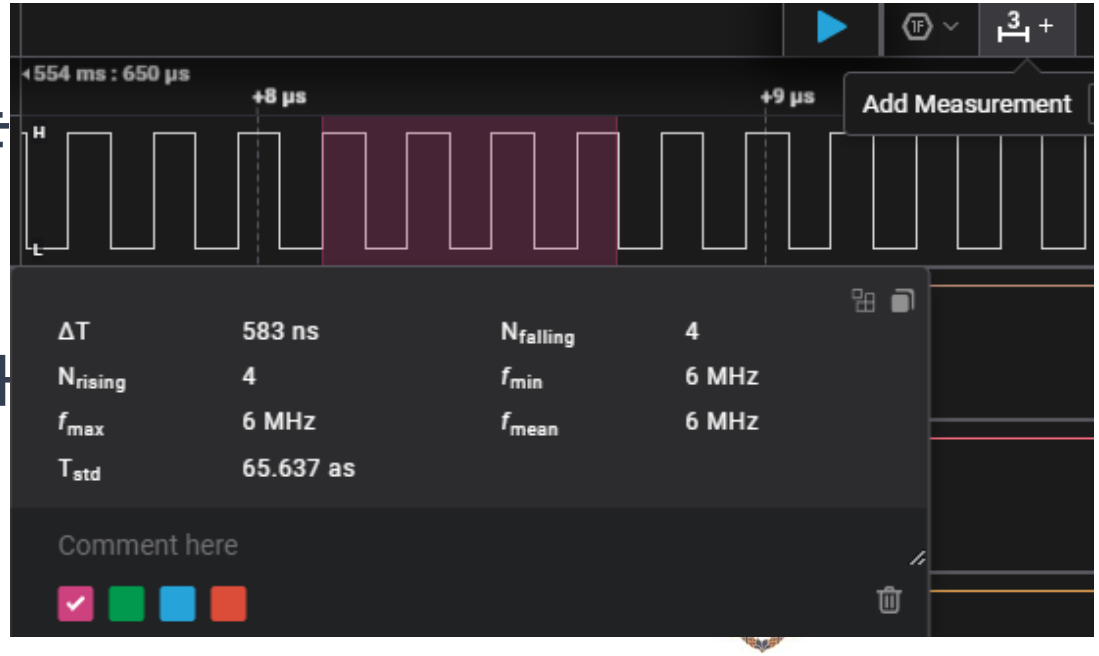
- 데이터 버스, 클럭, 통신 프로토콜(UART, SPI, I2C ...) 동작 분석이 가능함

- 신호의 타이밍, 상태 전환, 데이터 분석에 용이

- GDB는 소프트웨어 코드 레벨의 디버깅을, 로직 분석기는 하드웨어 레벨의 디버깅을 제공.

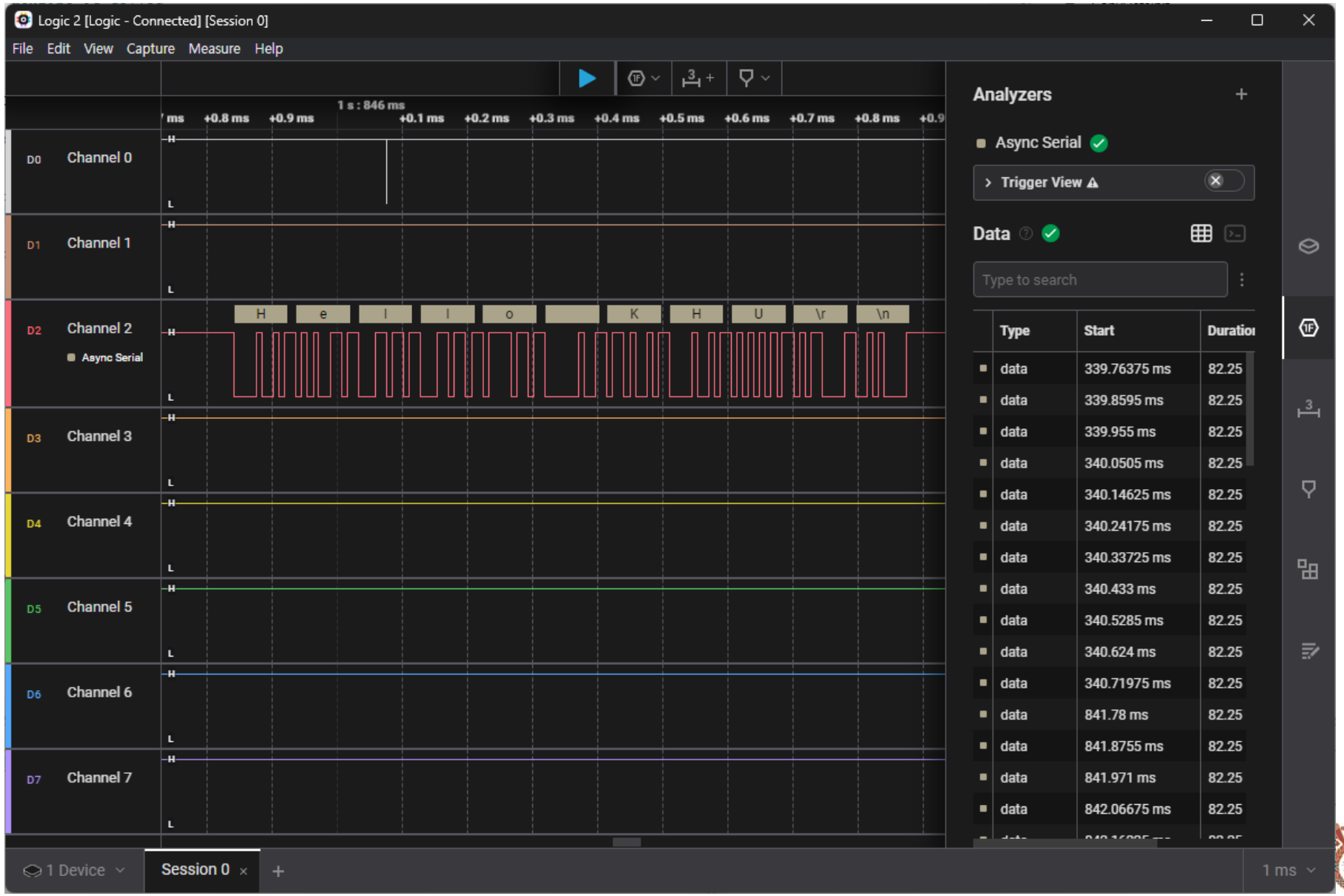
- 두 디버깅 방법을 적절히 활용하여 문제를 빠르게 파악하 해결이 가능함





Logic Analyzer

>> Logic Analyzer – UART Decode



Thank you

Department of Electronic Engineering,
Kyung Hee University, 1732, Deogyeong-Daero,
Giheung-gu, Yongin-si, Gyeonggi-Do, 17104,
Republic of Korea
Web: https://khu-hri.weebly.com/

Human-Robot Interaction Laboratory

KYUNG HEE UNIVERSITY